

19. Un choix, puis plusieurs

`&&` s'arrête dès que la réponse est connue.

1. Ce qu'il faut comprendre

`if ... else if ... else` enchaîne les cas. Rien que de très ordinaire — sauf la dernière ligne.

`bouton(B) && vies > 0` : si `bouton(B)` rend 0, **la partie droite n'est même pas calculée**. Ce n'est pas une finesse de compilation, c'est ce qui permet plus tard d'écrire `if (i < n && table[i] == 0)` sans lire une case hors du tableau.

Les fonctions de cette leçon — et le tuto de chacune

- `#include <texte>` → [0.0.2. La fonction texte\(\) — écrire un mot](#)
- `#include <bouton>` → [0.65.5. La fonction bouton\(\) — lire la manette](#)

2. Le programme, morceau par morceau

Le programme entier de la leçon, découpé comme l'œil le lit. Il fait **526 octets** de cartouche.

Ce que le compilateur y a gravé, nommé — chaque morceau, sa taille, qui l'a demandé — et ce qu'il a laissé, parce que rien ne l'appelle :

Gravé dans la cartouche :

```
démarrage      49 o ← la console, à l'allumage - la mise en route de la console
main()         83 o ← démarrage - ta fonction
VBlank         8 o ← la console, à chaque image - compte les images, 60 fois par seconde
AttendreImage  28 o ← image() - attend l'image suivante
AttendreVBlank 16 o ← démarrage - attend que l'écran soit libre
AttendreAcces  12 o ← EcrireTexte - attend de pouvoir écrire une case
EcrireTexte    10 o ← texte() - écrit un texte, lettre par lettre
CopierTuiles   85 o ← démarrage - copie les lettres et les dessins en mémoire vidéo
EffacerCarte   14 o ← démarrage - vide les deux cartes de l'écran
LireManette    37 o ← bouton() - lit les huit boutons
Tuiles         160 o ← CopierTuiles - 10 lettres sur 43
"PERDU  "      8 o ← texte() - un texte gravé
"DERNIERE"     8 o ← texte() - un texte gravé
"CA VA  "      8 o ← texte() - un texte gravé
```

Lettres gravées : ACDEINPRUV

Pas gravé (jamais appelé) : AvancerAirs, AttendreFinVBlank, AdresseCase, EffacerCases, EcrireNombre, Diviser, Reste, Multiplier, DecalerGauche, DecalerDroite, RangerLutins, InstallerTransfert, Hasard, EffacerFond, EffacerPanneau, DonneesTransfert, Notes

Fonctions du langage appelées :

```
texte() × 3    33 o sur place, dans main() + EcrireTexte (10 o)
bouton() × 1    5 o sur place, dans main() + LireManette (37 o)
image() × 1     3 o sur place, dans main() + AttendreImage (28 o)
```

1. CE QUE LE PROGRAMME RACONTE DE LUI-MÊME

```
// CE PROGRAMME : 3 vies. B en retire (tant qu'on le tient) ; un message
// différent selon qu'il reste plusieurs vies, une seule, ou aucune.
//
// if ... else if ... else : on essaie les cas dans l'ordre ;
// le premier qui est vrai est fait, les autres sont sautés.
// == veut dire « est égal à » (deux signes = ; un seul = veut dire « ranger dans »).

#include <texte>    // écrit un texte à l'écran
#include <bouton>    // lit un bouton de la manette
```

2. LE POINT D'ENTRÉE : C'EST ICI QUE LA CONSOLE ARRIVE

```
int main() {                                // Le jeu commence ici.
    uint8_t vies = 3;                        // On commence avec 3 vies.

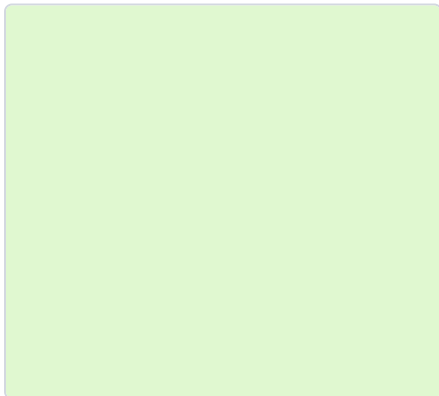
    while (true) {                           // Répète pour toujours :
        if (vies == 0)    texte(4, 8, "PERDU  "); // 0 vie : PERDU
        else if (vies == 1) texte(4, 8, "DERNIERE"); // sinon, 1 vie : DERNIERE
        else              texte(4, 8, "CA VA  "); // sinon (2 ou 3) : CA VA
                                                // (les espaces effacent les restes)

        if (bouton(B) && vies > 0) vies--;        // B tenue ET il reste des vies :
                                                // une de moins (« -- » retire 1).
                                                // Si bouton(B) vaut 0, la partie
                                                // droite n'est même pas regardée.
                                                // Attend l'image suivante.

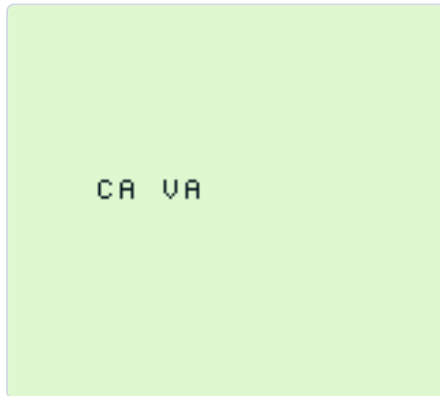
        image();
    }
}
```

3. Ce qui se passe, étape par étape

La cartouche a été compilée et exécutée pour de bon ; voici son écran aux instants qui comptent. Une image identique à la précédente n'est pas montrée — un programme qui ne bouge pas n'a qu'une seule image, et c'est la vérité sur ce programme.



la toute première image — la console vient de démarrer — image 1



un tiers de seconde plus tard : le premier dessin est posé — image 20

4. Ce que les touches changent

Chaque bouton que le programme lit a été enfoncé douze images, puis relâché. Voici l'écran juste après.



PERDU

B — l'écran a changé

5. Ce qu'on doit voir

« CA VA », puis « DERNIERE », puis « PERDU » — B après B.

6. Ce que la console répond

Ces contrôles ne sont pas décoratifs : ce sont ceux que `verifier-tuto.mjs` exige de cette leçon. Ils viennent d'être joués, et voici ce qu'ils ont mesuré.

- ✓ au départ, « CA VA »
- ✓ B fait tomber les vies à zéro (0)
- ✓ et « PERDU » s'affiche
- ✓ sans jamais déborder sous zéro

Ensuite : **20. Deux touches, deux effets** — chapitre 3 : Retenir, et réagir. · [gameboy3](#) — écrire une cartouche Game Boy en C++